



Reachability for airline networks: fast algorithm for shortest path problem with time windows

Xiaofeng Gao^a, Yueyang Xianzang^a, Xiaotian You^a, Yaru Dang^{b,*},
Guihai Chen^a, Xinglong Wang^b

^a Shanghai Key Laboratory of Scalable Computing and Systems, Department of Computer and Engineering, Shanghai Jiao Tong University, Shanghai 200240, China

^b Civil Aviation University of China, Tianjin 300300, China

ARTICLE INFO

Article history:

Received 22 March 2017

Received in revised form 5 December 2017

Accepted 19 January 2018

Available online 12 February 2018

Communicated by P.G. Spirakis

Keywords:

Complex network

Centrality

Reachability

Shortest path with time windows

ABSTRACT

Airline network, including airports as network nodes and flight routes as directed network edges, has a lot of special features such as departure and arrival times, air ticket budget, flight capacity, transportation cost, etc. Thus, analyzing network behavior and service performance for such a network is much more difficult than that for many other networks. In this paper, taking China domestic airline network as a representative, we try to discuss the reachability issue for each airport respectively, which could reflect its regional connectivity level and service quality of civil aviation. More specifically, we evaluate reachability through many features including node degree, betweenness, closeness, etc. To get the values of some features, we design a fast Dijkstra-based all-pair shortest path algorithm with both time and budget requirements, then use Fenwick Tree to further improve the time efficiency. Actually, it is a shortest path problem with time windows and other constraints. Furthermore, we propose a faster solution by reducing the edges in the duplicated graph as a simplification and then provide the time complexity proof. Finally, we implement Analytic Hierarchy Process (AHP) to convert the reachability feature into numerical values for all airports to measure their service qualities precisely. Our results for China domestic airline network with 210 airports and 69,160 flight routes will definitely become a guide to airline companies and civil aviation administration for their further development and management.

© 2018 Elsevier B.V. All rights reserved.

1. Introduction

Reachability and connectivity have been widely used as a measure to evaluate the networks and for graph there are also many widely used features to benchmark. However, when it now comes to the airline network, things will change. We consider an airline network as a graph including airports as nodes and flight routes as directed edges, which has a lot of differences from other networks. For instance, the graph may have hundreds or even thousands of parallel edges. Every edge in the graph has its own time window, which corresponds to the departure and arrival time of the flight route, and only in the time window can the edge be valid. What's more, every node has its specific values such as ground transportation

* Corresponding author.

E-mail addresses: gao-xf@cs.sjtu.edu.cn (X. Gao), xzyy0808@sjtu.edu.cn (Y. Xianzang), sjtuylxt@gmail.com (X. You), yarudang@cauc.edu.cn (Y. Dang), gchen@cs.sjtu.edu.cn (G. Chen), xinglong1979@163.com (X. Wang).

<https://doi.org/10.1016/j.tcs.2018.01.016>

0304-3975/© 2018 Elsevier B.V. All rights reserved.

cost, which makes all nodes essentially different. Since an airline network is a graph with a lot of distinctive features, it can hardly be measured in accordance with one or more explicit numerical indicators.

In this paper, we introduce a new definition of reachability, which is closely related to centrality and capacity. Centrality identifies how influential and important the node is in the graph, indicating the level of the corresponding airport. Capacity identifies the own value of the node itself regardless of the form of the graph. When it comes to centrality, we mainly consider degree centrality, betweenness centrality, and closeness centrality with time and budget requirements. For capacity, we consider flight frequency, seating capacity, and flight duration as auxiliary features. Then taking China airline network as a representative, we try to compute the reachability issue for each airport respectively through Analytic Hierarchy Process (AHP), which could indicate its rationality and superiority and be a guide to service improvement.

To figure out all the feature values, we design an algorithm to calculate the betweenness and closeness centrality, which is the most complicated part. There has been a fast algorithms for betweenness centrality [1], requiring $O(n + m)$ space and running in $O(nm + n^2 \log n)$ on a weighted graph. However as is stated above, airline network is a network with many realistic features, so we should compromise to some realistic constraints. Actually, it is a shortest path problem with time windows and other constraints. To solve this problem, we presume the time of transfers is at most 7 and the total journey will take the passenger at most 7 days, otherwise the journey is hardly seen in the realistic world. To meet the two demands, we convert the original graph to a new three-dimensional graph, of which the three ordinates denote the airport, the time of transfers and the time of that moment respectively. After adding the corresponding edges to the new graph, we use Dijkstra algorithm with min-priority queue [1] to solve the all-pair shortest path problem. Considering some unnecessary cases, we use Fenwick Tree to reduce the size of the states, which seemingly makes the time complexity worse but actually accelerate it a lot. Additionally, we construct another duplicated graph with less edges generated comparing to the duplicated graph and implement the simplest shortest path algorithm to provide a better solution.

To summarize, in this paper we present a novel way to measure the reachability issue of an airport in airline networks. Taking China Airline Network as a representative, we define eight features for evaluation, and use Analytic Hierarchy Process (AHP) with expert grading matrix to quantitatively evaluate the reachability of airports. We further design a fast detection method using Dijkstra-based algorithm with a min-priority queue and Fenwick tree to calculate the betweenness and closeness features in this network. Our results for China domestic airline network with 210 airports and 69,160 flight routes will definitely become a guide to airline companies and civil aviation administration.

The rest of this paper is organized as follows: Section 2 summarizes the related works in this area. Section 3 introduces the definitions and requirements of the problem. Section 4 designs a fast algorithm to detect the betweenness and closeness centrality. Section 5 refines the previous algorithm by constructing another duplicated graph with less edges. Section 6 discusses the rating procedure to evaluate the reachability issue of an airport. Finally, Section 7 gives the conclusion.

2. Related work

As for airline networks, Barros et al. [2] used DEA two-stage procedure to evaluate operational performance of European airlines, considering number of employees, number of planes and operational cost; Tsaur et al. [3] evaluated airline service quality by fuzzy MCDM considering tangibility, reliability, responsiveness, assurance and empathy; Yu et al. [4] used the SBM-NDEA model to assess the performance of airports considering the airside service and landside service; Bowen et al. [5] used Airline Quality Rating (AQR) Methodology to evaluate the US airline industry. Although most of these researches considered many factors of airline networks, surprisingly few of them took complex network theory into consideration.

As for centrality, Brandes et al. [1] designed a fast algorithm for betweenness centrality requiring $O(n + m)$ space and running in $O(nm + n^2 \log n)$ time on weighted networks; Kourtellis et al. [6] proposed a randomized algorithm for estimating betweenness centrality to identify nodes with high betweenness centrality; Lee et al. [7] constructed tourism-management strategies for villages by evaluating spatial centrality; Guimera et al. [8] claimed that nodes with high betweenness tend to play a more important role than those with high degree in the world-wide airport network. All of their work inspire us a lot, but we still need to tackle the centrality in such a complex network by ourselves.

As for the shortest path problem with time windows, some related works had time constraints on vertices but not edges. Mounpla et al. [9] gave a dynamic programming algorithm to solve shortest path with regular time windows like $[a_i, b_i]$ which means the vertex can only be visited between a_i and b_i ; Zhang et al. [10] gave an algorithm to solve shortest path with left-side time windows only. There are also some related works had time constraints on edges/paths. Chen et al. [11] proposed the MTT algorithm to solve time-constrained shortest path problem, and they assumed that there is no multiple arc, but our graph does have multiple edges between pairwise vertices; Ahuja et al. [12] proposed an algorithm to solve the minimum cost-path problems in street networks with periodic traffic lights. The fact that vehicles can pass intersections only when the traffic lights are green can be regarded as paths with time windows. After that, Androutsopoulos et al. [13] improved the solution to tackle a more general condition, and further gave their solution to solve the k -shortest path problem with time windows; Hu et al. [14] gave a better solution for the same problem recently. However, in our problem, which is in a more general and realistic condition, we should not only consider the least time period but also the least budget. That is a problem most of literatures not discussing about.



Fig. 1. A diagrammatic sketch to show the China airline network. (For interpretation of the references to color in this figure, the reader is referred to the web version of this article.)

3. Definitions and features

In this paper, we take China airline network as a representative to discuss the reachability issue for each airport. First, we construct China airline network according to weekly domestic flight statistics with 210 airports and 69,160 flight routes (data are collected from July 1st, 2016 to July 7th, 2016). Fig. 1 exhibits the connections between airports in mainland China with the help of JavaScript Visualization libraries like D3.js. In this figure, the blue nodes represent airports, while the grey links represent flight routes. Note that: (1) We focus on civil aviation and do not consider service aviation or cargo airline. Thus we do not plot Nanhai area. (2) We only record domestic flight routes and do not count flights from/to Taiwan, Hong Kong, and Macao. (3) Links in the figure only represents connectivity situation between airports (we omit the link direction for clarity), which do not reflect the real flight routes, since the domestic routes should be arcs within the territory of China. (4) We focus on regular flight routes, so we do not record the other irregular flight routes such as extra section flights.

Correspondingly, we construct a directed graph $G = (V, E)$. Every node in V denotes an airport while every edge in E corresponds to a flight route. For each $e \in E$, we denote the average air ticket price of this flight route as p_e , the number of seats on this flight (according to its aircraft type) as s_e , its departure time as d_e , and arrival time as a_e . For each $v \in V$, we denote its average ground transportation budget as b_v (denoting the average cost that city residents should spend to the airport) and the average ground transportation time as t_v .

To evaluate the reachability of an airport, we always need to detect the shortest path from one airport u to another v through graph G . In airline network, a path $\{u \rightarrow v \rightarrow w\}$ not only represents the connectivity property from u to w , but also denotes a valid flight transfer schedule for a passenger. Additionally, when considering a valid schedule, people care about the travel duration as well as the total budget. Thus, we denote the shortest path from airport u to airport v under the time and budget requirements respectively as follows.

Definition 1 (Shortest path with minimum time). A valid path from node u to v in an airline network G with Minimum time is a path $P = \{e_1, \dots, e_n\}$, where the tail of e_i should be the head of e_{i-1} and e_1 goes out of u , e_n goes into v respectively.

Additionally, $d_{e_i} \geq a_{e_{i-1}} + t$, for $i = 2, \dots, n$ and t is a flight transfer time. The shortest path with minimum time is a valid path such that the total time duration $a_{e_n} - d_{e_1}$ is minimized.

Let $SPT(u, v)$ denote the *shortest path with minimum time* for u and v . Easy to know, $|SPT(u, v)|$ is the path length (or cardinality of the edges) and to be realistic, $|SPT(u, v)| \leq K$ where K is the maximum transfer number (usually $K \leq 7$), and usually $t \geq 90$ mins according to a regular flight transfer procedure. For convenience, we denote the time on the path $SPT(u, v)$ i.e. $a_{e_n} - d_{e_1}$ as $spt(u, v)$.

Similarly, if we consider the *shortest path with minimum budget* for u and v , instead of computing $a_{e_n} - d_{e_1}$, we want to minimize $\sum_{i=1}^n p_{e_i}$ with $a_{e_n} - d_{e_1} \leq T$ where T is the upper bound of total travel time duration (usually $T \leq 7$ days = 10080 minutes) and we use $SPB(u, v)$ to denote such a path and $spb(u, v)$ to denote the budget i.e. $\sum_{i=1}^n p_{e_i}$.

To analyze the reachability of a node u , we need to comprehensively consider the connectivity and importance of this airport. With the help of complex network theory, we select several features of u and evaluate the level of reachability quantitatively. These features are explained as follows.

Definition 2 (Clustering coefficient [15]). The clustering coefficient of a node u (CC_u) in an airline network is the portion of the pairs of connected nodes (of which the number is k_u) within its neighborhood divided by the maximal possible edges ($k_u(k_u - 1)$) between them, written as:

$$CC_u = \frac{1}{k_u(k_u - 1)} \sum_v \sum_w I(u, v)I(u, w)I(v, w)I(w, v), \quad (1)$$

where $I(u, v)$ is presented as an indicator:

$$I(u, v) = \begin{cases} 1, & \exists e \in E, \text{ s.t. } e \text{ is from } u \text{ to } v; \\ 0, & \text{otherwise.} \end{cases} \quad (2)$$

Degree centrality [16], betweenness centrality [17] and closeness centrality [18] are used as three major measures to evaluate a network node. Degree centrality symbolizes the importance of the node in a network, while betweenness centrality measures the extent to which a particular node lies between other nodes in a network and closeness centrality indicates the distance from all other nodes. Their definitions are represented as follows.

Definition 3 (Degree centrality). Degree centrality of a node u (DC_u) in an airline network is the portion of connected nodes by all the other nodes. In other words, it is the ratio of through-flight, written as:

$$DC_u = \frac{\sum_v I(u, v)}{|V| - 1}. \quad (3)$$

Definition 4 (Betweenness centrality with time and budget requirements). Betweenness centrality of a node u (BC_u) in an airline network is defined as the ratio of all shortest paths (with time and budget requirements) passing through it and reflects its transitivity. More formally,

$$BC_u = \frac{\sum_{v \in E \wedge v \neq u} \sum_{w \in E \wedge w \neq u \neq v} (I_t(u, v, w) + I_b(u, v, w))}{2 \cdot (|V| - 1)(|V| - 2)}, \quad (4)$$

where $I_t(u, v, w)$ and $I_b(u, v, w)$ are indicators defined as Eqn. (5) and Eqn. (6).

$$I_t(u, v, w) = \begin{cases} 1, & \exists e_i \in SPT(v, w), \text{ s.t. } e_i \text{ is from } u; \\ 0, & \text{otherwise.} \end{cases} \quad (5)$$

$$I_b(u, v, w) = \begin{cases} 1, & \exists e_i \in SPB(v, w), \text{ s.t. } e_i \text{ is from } u; \\ 0, & \text{otherwise.} \end{cases} \quad (6)$$

Definition 5 (Closeness centrality of time). Closeness Centrality of Time of a node u (CCT_u) is the sum of the average time needed on $SPT(u, v)$ ($spt(u, v)$) for v among all the other nodes and the ground transportation time. More formally,

$$CCT_u = \frac{\sum_{v \in V \wedge v \neq u} spt(u, v)}{|V| - 1} + t_u. \quad (7)$$

Definition 6 (*Closeness centrality of budget*). Closeness Centrality of Time of a node u (CCB_u) is the sum of the average budget needed on $SPB(u, v)$ ($spb(u, v)$) for v among all the other nodes and the ground transportation budget. More formally,

$$CCB_u = \frac{\sum_{v \in V \wedge v \neq u} spb(u, v)}{|V| - 1} + b_u. \quad (8)$$

Considering a node u , we define the *centrality* feature of u (CT_u) comprehensively in Eqn. (9), combining clustering coefficient, degree centrality, betweenness centrality, and closeness centrality together.

$$CT_u = f_{CT}(CC_u, BC_u, DC_u, CCT_u, CCB_u). \quad (9)$$

Next, we consider the specific features for airline networks and give Definition 7–9.

Definition 7 (*Flight frequency*). The Flight Frequency of a node u (FF_u) is the amount of the edges from/to u . More formally,

$$FF_u = \sum_{e \in E} (I_{in}(u, e) + I_{out}(u, e)), \quad (10)$$

where $I_{in}(u, e)$ and $I_{out}(u, e)$ are defined as follows:

$$I_{in}(u, e) = \begin{cases} 1, & e \text{ is to } u; \\ 0, & \text{otherwise.} \end{cases} \quad I_{out}(u, e) = \begin{cases} 1, & e \text{ is from } u; \\ 0, & \text{otherwise.} \end{cases} \quad (11)$$

Definition 8 (*Seating capacity*). The Seating capacity of a node u (SC_u) is the average number of the seats on the flight from the airport. More formally,

$$SC_u = \frac{\sum_{e \in E} I_{out}(u, e) s_e}{\sum_{e \in E} I_{out}(u, e)}. \quad (12)$$

Definition 9 (*Flight duration*). The Flight Duration of a node u (FD_u) is the average duration of the flight routes from the airport. More formally,

$$FD_u = \frac{\sum_{e \in E} I_{out}(u, e) (a_e - d_e)}{\sum_{e \in E} I_{out}(u, e)}. \quad (13)$$

For a node u , we can define the *capacity* property of u (CP_u) comprehensively as shown in Eqn. (14).

$$CP_u = f_{CP}(FF_u, SC_u, FD_u). \quad (14)$$

Finally we can define the *reachability* of a node u (R_u) as follows:

$$R_u = f_R(CT_u, CP_u). \quad (15)$$

Note that functions f_{CT} in Eqn. (9), f_{CP} in Eqn. (14) and f_R in Eqn. (15) can be customized according to different airline network architectures and requirements.

4. Fast detection of betweenness and closeness

To figure out betweenness centrality and closeness centrality, we must solve the *all-pair shortest path* problem for not only the optimal $spt(u, v)$ and $spb(u, v)$ but also the trajectory of all the paths. As we mentioned above, G is a graph with multiple edges and loops and every edge has a time window so that the existing algorithms such as Dijkstra Algorithm cannot solve the problem with the transfer and duration constraints.

To implement Dijkstra-based algorithm, we need to first convert the airline network G into a simple directed graph G' . Note that a valid path links one edge to another only when the two edges are connected and their corresponding flight route can be transferred successfully. Thus the connection in the new generated graph should not only represent the connectivity issues, but also reflect the time and transfer hop information. Motivated by this observation, we introduce *duplicated graph* generated from G as follows:

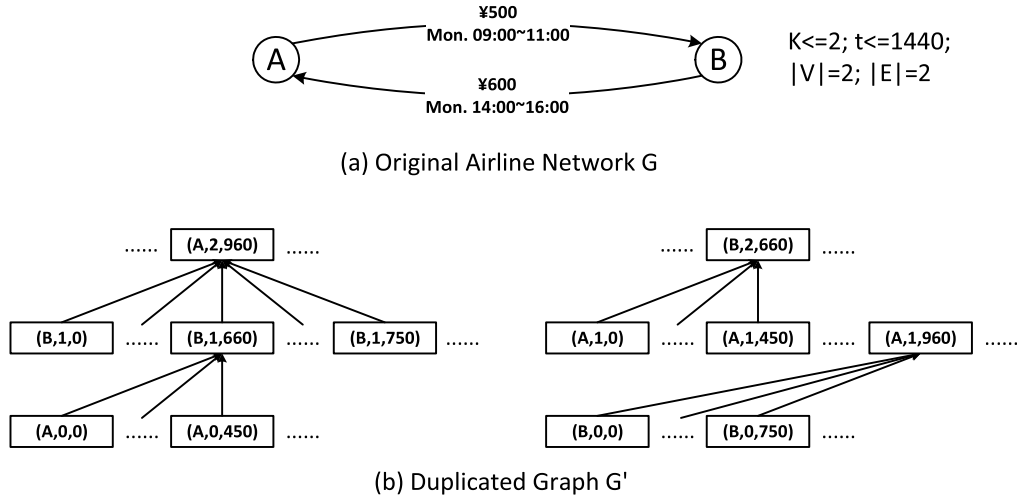


Fig. 2. An example duplicated graph G' generated from G .

Definition 10 (*Duplicated graph*). The duplicated graph of G is a simple directed graph $G' = (V', E')$ where a node in V' is denoted as (v, k, t) and an edge in E' from v'_1 to v'_2 is denoted as (v'_1, v'_2) . If there is a node $v' = (v, k, t) \in V'$, it means that at time t (take minutes as the unit) the passenger reach the airport v after the k^{th} transfer. $\forall e \in E$ (from u to v), there is an edge $e' = ((u, k, t), (v, k + 1, a_e))$ with weight $w_{e'} = p_e$ (with $k + 1 \leq \text{transfer constraints}$ and $t \leq d_e - 90$).

In G' , k reflects the transfer number, while t reflects the time information. In reality, we usually set $k \leq K$ and $t \leq T$.

Obviously every path in G' corresponds to a valid path in G and for every valid path in G , there must exist at least one corresponding path in G' . If we solve the all-pair shortest path problem on G' , we can easily figure out betweenness centrality and closeness centrality in the original graph.

Fig. 2 is an example to illustrate this conversion. Fig. 2 (a) is a simple flight network G with two nodes A and B , as well as two flight routes both departure on Monday. One is flight $A \rightarrow B$ with duration 09:00–11:00, and another is $B \rightarrow A$ during 14:00–16:00. We set $k \leq 2$ (means we only allow less than two transfers), and $0 \leq t \leq 1440$ (means the trip should be finished within one day). Fig. 2 (b) is the converted graph G' . Since the first flight departs at 09:00, nodes $(A, 0, 0)$ to $(A, 0, 450)$ are all valid to reach $(B, 1, 660)$ since the flight departs at 09:00, so the latest valid boarding time should be $9 \times 60 - 90 = 450$. Next, the flight takes 120 mins and reaches B at 11:00, so the t value at B part should be 660. (There are totally 451 nodes if we count per minutes.) Each of the edge has weight ¥500 as ticket cost. Similarly, nodes labeled $(B, 0, 0)$ to $(B, 0, 750)$ all link to $(A, 1, 960)$ if we departure at $14 \times 60 \text{ min} - 90 \text{ min} = 750 \text{ min}$.

Easy to see, G' has two connected components depicting all the possible transfer plans. $|V'| = 2 \times 3 \times 1441 = 8646$ and $|E'| = (451 + 751) \times 2 = 2404$ edges (but obviously there are only 4 vertices with positive in-degree, which is the key observation to reduce the algorithm complexity).

The next step is to solve the shortest path problem in the new graph. As a simple graph with non-negative weights (on edges as budget), we implement Dijkstra algorithm [19]. Initially we choose a $v \in V'$ and start from this specific source. At each step, we pick the closest node in the set of undiscovered nodes, update the shortest path to each undiscovered node and remove the node from the set. We keep doing it until every node is picked. Here we use array $path[u]$ to record the in-edge of the shortest path from the starting node to u , and use array $prev[u]$ to record u 's previous hop along the path.

The final step is to find the corresponding shortest path in the original graph. We have got the minimum distance from the source to all other nodes with specific time and specific transfer numbers, so we can easily get the minimum duration and the minimum budget needed from the source to all the nodes. By tracing down $path[\cdot]$ and $prev[\cdot]$ arrays, we can easily get the corresponding shortest paths in original graph G .

Note that for G' , there are only $O(|E|)$ nodes with positive in-degree (can be found from Fig. 2), so there are $O(|E|)$ valid nodes, i.e. $|V'| = O(|E|)$. For each node, there are at most $O(|E|)$ edges from it, so there are $O(|E|^2)$ edges i.e. $|E'| = O(|E|^2)$. If we compute all-pair shortest path with Dijkstra algorithm, the total time complexity is $O(|V'|^3) = O(|E|^3)$. Apparently, for China airline network, the algorithm can hardly hit the upper bound of the time complexity. However, it still works with extremely low efficiency as a matter of fact. We choose to invoke min-priority queue [20] to improve the time efficiency. The detailed description is shown in Algorithm 1. Here $dist[u]$ is the array to record the minimum distance from the starting node to u . Actually, we need to run Algorithm 1 on G' for $|E|$ times to compute all-pair shortest path (starting from the representative of the departure point of each flight route in G).

After adding min-priority queue, the time complexity (with Fibonacci heap) is still $O(|V'|^2 \log |V'| + |V'| |E'|) = O(|E|^3)$, but later we will show how it improves the time efficiency via numerical experiments. Actually, we can further optimize the efficiency of this algorithm. Now we focus on the distribution of the shortest distances to give the following lemma:

Algorithm 1: Dijkstra algorithm with priority queue.

Input: the graph G' , the source s .
Output: the set of minimum distance $dist[\cdot]$, the set of previous nodes $prev[\cdot]$, the set of previous path $path[\cdot]$.

```

1 Create vertex min-distance priority queue  $Q$ ;
2 foreach vertex  $v$  in  $V'$  do
3    $dist[v] \leftarrow +\infty$ ;  $prev[v] \leftarrow \emptyset$ ;  $path[v] \leftarrow \emptyset$ ;           // initialization
4    $Q \leftarrow push(Q, v)$ ;                                           // push  $v$  into queue  $Q$ 
5  $dist[s] \leftarrow 0$ ;
6 while  $Q$  is not empty do
7    $u \leftarrow pop(Q)$ ;                                           // pop out the first vertex from  $Q$ 
8   foreach edge  $e$  from  $u$  to  $v$  do
9      $alt \leftarrow dist[u] + w_e$ ;
10    if  $alt < dist[v]$  then
11       $dist[v] \leftarrow alt$ ;  $prev[v] \leftarrow u$ ;  $path[v] \leftarrow e$ ;

```

Lemma 1. $\forall v \in V, \forall k \leq K, \forall t_2 < T$, if $\exists t_1 < t_2$ s.t. $dist[(v, k, t_1)] < dist[(v, k, t_2)]$, then all the results (minimum time/budget path) will not change if we delete the node (v, k, t_2) .

Proof. To simplify the problem, assume now we are exploring the shortest paths from a source $(s, \cdot, \cdot)$ to all other nodes with time and budget requirements. Let $x, y \in [0, 1]$, then what we want to minimize is a linear combination of flight time and budget, shown as $x \cdot time + y \cdot budget$. Especially in this paper we choose $x = 0, y = 1$ (as minimum budget) and $x = 1, y = 0$ (as minimum time).

Now, given a t_2 , if $\exists t_1 < t_2 < T$, we have $dist[(v, k, t_1)] < dist[(v, k, t_2)]$, then we can get that

$$x \cdot t_1 + y \cdot dist[(v, k, t_1)] < x \cdot t_2 + y \cdot dist[(v, k, t_2)],$$

which means the path starting from $(s, \cdot, \cdot)$ and ending up at node (v, k, t_2) can never be an answer to any requirement.

Next, let us check other paths passing through (v, k, t_2) . Assume there is a shortest path starting from $(s, \cdot, \cdot)$ and passing from (v, k, t_2) to $(v', k + 1, t_3)$, denoted as $P_1 = \{(s, \cdot, \cdot) \rightarrow \dots \rightarrow (v, k, t_2) \rightarrow (v', k + 1, t_3) \rightarrow \dots\}$, then we can change it to $P_2 = \{(s, \cdot, \cdot) \rightarrow \dots \rightarrow (v, k, t_1) \rightarrow (v', k + 1, t_3) \rightarrow \dots\}$. By Definition 10, we know that there must exist an edge from (v, k, t_1) to $(v', k + 1, t_3)$ with the same weight as (v, k, t_2) 's and $dist[(v, k, t_1)] < dist[(v, k, t_2)]$. Then the length of P_2 is shorter than that of P_1 , which violates the shortest property of P_1 , a contradiction! Hence node (v, k, t_2) will not be involved in any shortest path starting from $(s, \cdot, \cdot)$. Additionally, since $dist[(v, k, t_1)] < dist[(v, k, t_2)]$, (v, k, t_1) can be picked earlier than (v, k, t_2) in Algorithm 1. Therefore, (v, k, t_2) can never be used to update the $dist[\cdot]$ value for all other nodes.

In conclusion, (v, k, t_2) can never be an answer to any requirement and it can never be used to update the $dist[\cdot]$ value of all other nodes, so it is safe to remove node (v, k, t_2) when exploring the shortest path from starting point $(s, \cdot, \cdot)$. $\square$

Inspired by Lemma 1, whenever we select a node $(v, k, t) \in V'$, we can first check whether there exists another node v' with earlier t' and smaller $dist[v']$ value. To achieve this purpose, we use a Fenwick tree [21] to calculate the prefix minimum value as $F[v][k][\cdot]$, which is defined in Definition 11.

Definition 11 (Fenwick tree [21]). A Fenwick Tree (in our algorithm), denoted as $F[\cdot][\cdot][\cdot]$, is a data structure to calculate the prefix minimum value. More formally, $\forall v \in V, \forall k \leq K, \forall t \leq T$,

$$F[v][k][t] = \min_{0 \leq i \leq t} dist[(v, k, i)].$$

There are two operations on the Fenwick Tree: insert a new value (alt) into the old minimum prefix (denoted as $F[v][k][t] \leftarrow alt$) and calculate the minimum prefix (just denoted as $F[v][k][t]$). Both of them occupies $O(\log T)$ time, where T is the time range (in our examples $T = 10080$).

For a node (v, k, t) in G' , whenever we plan to update the $dist[\cdot]$ of all its neighbors (like Line 8 to Line 11 in Algorithm 1), we will compare $dist[(v, k, t)]$ to the prefix minimum value of $F[v][k][t]$. If $dist[(v, k, t)] > F[v][k][t]$, it means we meet a case satisfying Lemma 1, and thus (v, k, t) is redundant for the shortest path exploration. We can safely remove this node without influencing the final result, which will obviously reduce the intermediate operations for computation. Note that once we change a $dist[\cdot]$ value, we need to modify the Fenwick tree in $O(\log T)$ time. Thus we get an optimized algorithm shown in Algorithm 2.

Theorem 1. The **if** statement in Line 9 of Algorithm 2 will not change the minimum time or minimum budget we finally calculate.

Algorithm 2: Dijkstra algorithm with priority queue and Fenwick tree.

Input: the graph G' , the source s .
Output: the set of minimum distance $dist[\cdot]$, the set of previous node $prev[\cdot]$, the set of previous path $path[\cdot]$.

```

1 create vertex min-distance priority queue  $Q$ ;
2 create min-prefix Fenwick tree  $F[\cdot][\cdot][\cdot]$ ;
3 foreach vertex  $v$  in  $V'$  do
4    $dist[v] \leftarrow +\infty$ ;  $prev[v] \leftarrow \emptyset$ ;  $path[v] \leftarrow \emptyset$ ;
5    $Q \leftarrow push(Q, v)$ ;                                     // push  $v$  into queue  $Q$ 
6  $dist[s] \leftarrow 0$ ;
7 while  $Q$  is not empty do
8    $u = (v_1, k_1, t_1) \leftarrow pop(Q)$ ;                       // pop out the first vertex from  $Q$ 
9   if  $dist[u] \leq F[v_1][k_1][t_1]$  then                       // Compare  $u$  with the min value in  $F$  and decide whether to update  $dist[\cdot]$  for  $u$ 's
    neighbors
10    foreach edge  $e$  from  $u$  to  $v = (v_2, k_2, t_2)$  do
11       $alt \leftarrow dist[u] + w_e$ ;
12      if  $alt < dist[v]$  then
13         $dist[v] \leftarrow alt$ ;  $F[v_2][k_2][t_2] \leftarrow alt$ ;  $prev[v] \leftarrow u$ ;  $path[v] \leftarrow e$ ;

```

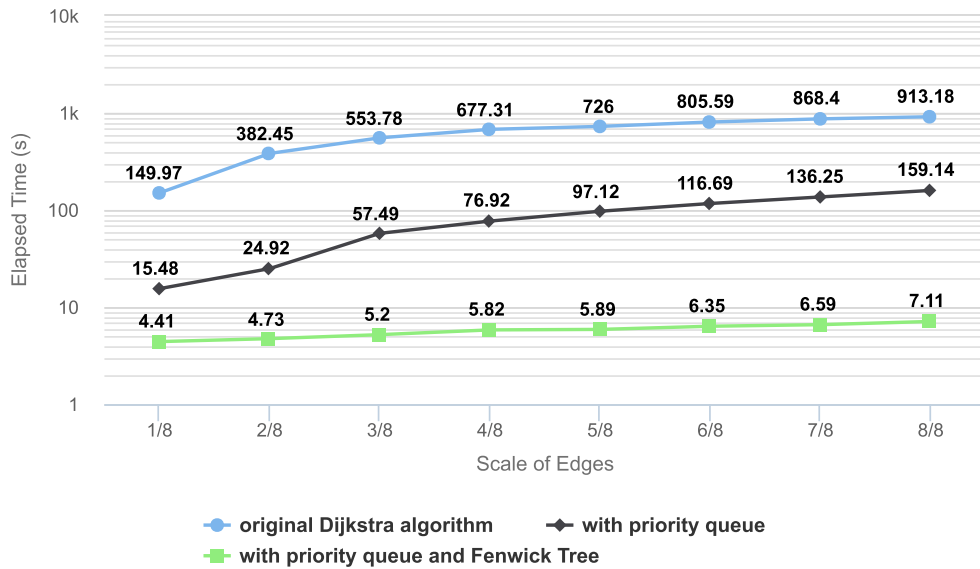


Fig. 3. A comparison among three algorithms.

Proof. When $dist[u] \leq F[v_1][k_1][t_1]$ is false, we can get

$$dist[(v_1, k_1, t_1)] > \min_{0 \leq i \leq t_1} dist[(v_1, k_1, i)].$$

That means $\exists t_0 < t_1$ s.t. $dist[(v_1, k_1, t_0)] < dist[(v_1, k_1, t_1)]$ and by Lemma 1, all the results will not change if we delete u . $\square$

By Theorem 1, Algorithm 2 turns out the right answers. After adding Fenwick tree, the complexity of the algorithm seemingly turns even worse to $O(|E|^3 \log T)$. However, in practice we find that Fenwick tree improves the algorithm more than ten times faster, and Fig. 3 exhibits this phenomenon. We randomly choose 2 nodes as the sources and randomly choose different scales of edges, use these three algorithms to detect the shortest paths to all the other nodes, repeat three times, record the average elapsed time, and plot a chart for comparison. The x-axis denotes the scale of edges, while the y-axis is the logarithmic value of computing time. From this figure we can see clearly that the two optimization strategies improve the practical performance efficiency greatly.

This successful improvement indicates that in realistic airline networks traditional shortest path algorithm will detect in a non-human way, which sometimes seems ridiculous. For instance, if we want to get a solution with the least budget from Beijing to Shanghai, we will never consider first flying to Alaska then flying to Shanghai, but in those traditional algorithms we have to detect such paths because meanwhile we cannot know whether the stupid path will be a subpath of the shortest path until we figure out all the solutions. The Fenwick Tree is to cut the waste of time of detection on such stupid path

Algorithm 3: Construct new graph.

Input: $G = (V, E)$, p_e , d_e , a_e
Output: $G^S = (V^S, E^S)$

- 1 Create arrival and departure vertices set A_i , D_i for every airport v_i ;
- 2 **foreach** $e = (v_i, v_j) \in E$ **do**
- 3 create two vertices v_i^{ed} , v_j^{ea} ;
- 4 create a direct edge e^S from v_i^{ed} to v_j^{ea} whose weight is p_e ;
- 5 $A_j \leftarrow \{v_j^{ea}\}$, $D_i \leftarrow \{v_i^{ed}\}$;
- 6 **foreach** $v_i \in V$ **do**
- 7 **foreach** $v_i^{ea} \in A_i$ **and** $v_i^{ed} \in D_i$ **do**
- 8 **if** passengers can transfer from flight $m(v_i^{ea})$ to flight $m(v_i^{ed})$ **then**
- 9 create a direct edge from v_i^{ea} to v_i^{ed} with weight 0;

during the detection. Obviously the correctness is based on Theorem 1 and the efficiency is based on a common sense which is the more time we need, the less budget we spend, because we will never pay more money on a solution taking more time.

All of the improvement is based on the features in realistic airline networks. If the features do not exist any longer, the algorithm will even be slower than the traditional algorithms. We can easily construct a graph satisfying the condition: Just make the prices lower when the distances and durations become longer in the graph, where our algorithm will get a bad performance. However, we often pay more to fly farther, so our algorithm always get a better performance.

5. A simplified construction for duplicated graph with faster solution

The reason that the above Dijkstra based algorithm is time consuming is mainly because the converted duplicated graph has too many dummy edges. Observing that we can actually construct one edge taking charge of multiple edges without violating the connectivity property, we can convert the original airline network into a simpler duplicated graph to better solve this problem. Before the transformation, let us first introduce some preliminaries.

Definition 12 (*State vertex*). A **state vertex** v represents the status of a flight at an airport, e.g., an arrival or a departure.

Using state vertex, we can add bunch of vertices at each airport to depict the incoming or outgoing flight edges. Next, we can describe a valid transfer for passengers according to Definition 13.

Definition 13 (*State transfer edge*). A **state transfer edge** $e = (a, b)$ is a direct weighted edge from a state vertex a to another state vertex b , meaning that passengers are available to transfer from a (maybe an arrival of a flight) to b (maybe a departure of another flight). The weight of this edge $W(e)$ denotes the expense of passengers during this transfer.

With the help of Definition 12 and Definition 13, we can now construct another duplicated graph to eliminate the influence of the time window constraint so that we can implement the traditional Dijkstra based algorithm to solve the problem. First, we create two state vertices for every flight, one of which represents its departure, while the other represents its arrival. Obviously, we can transfer from the departure vertex to arrival vertex by taking this flight, so we create a direct edge from the departure vertex to the arrival vertex with a weight as the price of the flight.

Next, we create state transfer edges to represent flight transfers. If passengers can transfer from flight A to flight B (satisfying the required time constraint), we can create an edge from the arrival vertex of A to the departure vertex of B , whose weight is 0.

By converting every flight into pairwise departure and arrival state vertices (with corresponding state transfer edges), now we successfully replace the original airline network into a duplicated graph, and, of course, with much less edges comparing with the previous transformation in Definition 10 of Sec. 4. Giving an airline network $G = (V, E)$ with all information, let $G^S = (V^S, E^S)$ denote the constructed graph, Algorithm 3 shows the details of this conversion. To better describe the algorithm, we define a projection function $m(v_i^S)$ to denote the original flight edge $e = (v_i, v_j)$.

Fig. 4 exhibits a toy example for Algorithm 3 with 4 vertices and 7 edges in the original airline network (shown as the left graph). After conversion process, the new graph G^S will have 14 vertices and 15 edges, reflecting all valid transfer schedule for passengers.

There is one more constraint that passengers should not take flights for more than K times. To reflect this constraint in G^S , we can image it as a conceptual duplicated graph H^S where node is denoted as (v^S, k) , meaning that passengers arrive at vertex v^S by taking k flights ($k \leq K$). Actually, we do not need to construct it in practice, but only modify the intermediate status of distance function when calculating the shortest paths using Dijkstra based algorithm. For example, if (u, v) is in G^S where u is a departure vertex and v is an arrival vertex, then for every k , $((u, k), (v, k+1))$ is in H^S . If (u, v) is in G^S where u is an arrival vertex and v is a departure vertex, then for every k , $((u, k), (v, k))$ is in H^S .

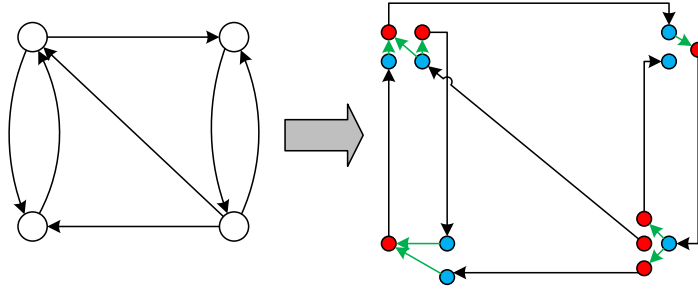


Fig. 4. An illustration for Algorithm 3. In this graph, blue vertices are arrival vertices, red vertices are departure vertices, green edges account for transfer and black edges account for flights. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

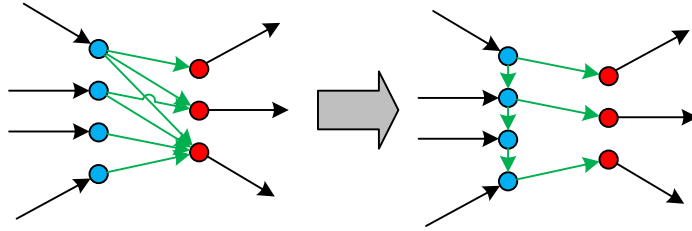


Fig. 5. An illustration of reducing edges. (For interpretation of the references to color in this figure, the reader is referred to the web version of this article.)

We know that the complexity of Dijkstra Algorithm with priority queue for a graph $G = (V, E)$ is $O(|E| \log |V|)$. Thus, we need to calculate the edges numbers and vertices numbers of H^S .

Lemma 2. *There are $2(K + 1)|E|$ vertices in graph H^S .*

Proof. From Algorithm 3 we know that $2|E|$ vertices are created in graph G^S . H^S has nodes (v, k) for every v in G and k from 0 to K . Thus, the numbers of vertices in graph H^S is $2(K + 1)|E|$. $\square$

Now, we separate the vertices in H^S into $K + 1$ sets $\{V_i^S\}$ where i is from 0 to K . Each set V_i^S contains the vertices (v, i) for every v in G^S .

Lemma 3. *There are $K|E|$ edges between $\{V_i^S\}$ in graph H^S .*

Proof. An edge between $\{V_i^S\}$ account for a flight. Since there are $|E|$ flights in graph G^S , there are $K|E|$ edges between $\{V_i^S\}$ in graph H^S . $\square$

However, it is hard to calculate the number of edges within $\{V_i^S\}$. We will not give an upper bound of it, instead we will try to reduce the edges within $\{V_i^S\}$ and give the exact number of inner edges in each V_i^S .

Suppose there is an airport v_i with the arrival vertices set A_i . We first sort A_i by the vertices' corresponding flight arrival times. Suppose the sorted list is $\{v_1^a, v_2^a, \dots, v_n^a\}$. Obviously, passengers can always transfer to a flight if they arrive earlier. Therefore, the set of departure vertices to which v_{i+1}^a links is a subset of v_i^a 's.

We only consider the connectivity between arrival vertices and departure vertices of a same airport. Thus, we remove the links from v_i^a to a departure vertex u if v_{i+1}^a also links to it. Then we add links from v_i^a to v_{i+1}^a for every i .

Fig. 5 shows a reduction phase to eliminate redundancy from H^S . The left graph exhibits a scenario at one airport, where the arrival vertices are sorted from top to bottom in ascending order to their arrival time. Initially, they will be connected to all possible departure vertices satisfying the transfer time constraint. Next, since the latter arrival vertex's available transfer plan can always be accepted by its earlier prior flight, we can remove some edges and add a new edge from earlier arrival vertex to later arrival vertex, which still satisfies the connectivity property whereas keeps the transfer constraint. As shown in the right graph of Fig. 5, we remove two green edges of the top blue vertex while add a new edge pointing to the second arrival vertex.

Correspondingly, we can provide a new algorithm to construct G^S , as shown in Algorithm 4.

Lemma 4. *The number of edges in $A_i \cup D_i$ of airport v_i in G^S is $O(2|E|)$, which means they are valid flight transfer plan or waiting slot to merge with the second available flight.*

Algorithm 4: Construct reduced graph.

Input: $G = (V, E)$, p_e , d_e , a_e
Output: $G^s = (V^s, E^s)$

- 1 Create arrival and departure vertices set A_i , D_i for every airport v_i ;
- 2 **foreach** $e = (v_i, v_j) \in E$ **do**
- 3 create two vertices v_i^{ed} , v_j^{ea} ;
- 4 create a direct edge e^s from v_i^{ed} to v_j^{ea} whose weight is p_e ;
- 5 $A_j \leftarrow \{v_j^{ea}\}$, $D_i \leftarrow \{v_i^{ed}\}$;
- 6 **foreach** $v_i \in V$ **do**
- 7 **while** $A_i \neq \emptyset$ and $D_i \neq \emptyset$ **do**
- 8 $v_a = \text{argmin}_i A_i$;
- 9 $v_d = \text{argmin}_i D_i$;
- 10 **if** $v_c = \text{argmin}_i (A_i - \{v_a\})$ exists $\wedge$ passengers can transfer from $m(v_c)$ to $m(v_d)$ **then**
- 11 $A_i = A_i - \{v_a\}$;
- 12 **else**
- 13 $E^s \leftarrow (v_a, v_d)$; $D_i = D_i - \{v_d\}$;
- 14 Create direct edges from every vertices in A_i to its next vertex if it has;

Algorithm 5: Selecting valid paths between two airports.

Input: Connectivity between D_i and A_j
Output: the valid paths set S

- 1 add a virtual vertex v_l with a very large arrival time in A_j ;
- 2 **while** $D_i \neq \emptyset$ and $A_j \neq \emptyset$ **do**
- 3 $v_d = \text{argmax}_i D_i$;
- 4 $v_a = \text{argmax}_i A_j$;
- 5 **while** $v_c = \text{argmax}_i (A_j - \{v_a\})$ exists $\wedge v_d$ can reach to v_c **do**
- 6 $A_j = A_j - \{v_a\}$;
- 7 **if** v_d can reach to v_a **then**
- 8 $S \leftarrow \text{Path}(v_d, v_a)$;

Proof. From Algorithm 4 we know that every departure vertex is pointed at most once by arrival vertices at a same airport. These arrival vertices form a chain, each of which can be pointed at most once as well. Therefore, the number of edges is not larger than the number of vertices, which is $2|E|$. $\square$

Edges reduced from V_i^s is the same as the edges in $A_i \cup D_i$ of airport v_i in G^s for every i . Thus, the number of these edges is $O(2(K+1)|E|)$. There are $2(K+1)|E|$ vertices and $O(2(K+1)|E|)+K|E|$ edges in H^s , so the complexity of Dijkstra Algorithm is $O(K|E|\log K|E|)$.

Suppose we get shortest paths for every vertices pair (v_i^s, v_j^s) where v_i^s is a departure vertex and v_j^s is a arrival vertex. The shortest paths from airport v_i to airport v_j , however, are not the union of the shortest paths from $v_i^s \in D_i$ to $v_j^s \in A_j$.

If passengers want to minimize their cost, we simply choose the minimal cost one among shortest paths from $v_i^s \in D_i$ to $v_j^{sl} \in A_j$ which has the latest arrival time.

If passengers want to minimize their travel time, an obvious fact is that a valid minimal time path should not depart earlier but arrive later. We will use this fact to select valid paths. We infer connectivity between D_i and A_j . Suppose D_i and A_j are sorted ascendingly. Here is the algorithm to select the most appropriate path, shown in Algorithm 5.

Finally, we will analyze the complexity of constructing algorithm and selecting algorithm. Suppose we use the typical Mergesort algorithm to sort list. In the first part of Algorithm 4, we operate $O(1)$ for every edges in the origin graph. In the second part of Algorithm 4, we sort all A_i and D_i for every airport and visit every vertex once, execute $O(1)$ operations as well. Therefore, the complexity is $O(|E| + |V|\log|E|)$.

We visit every vertex for once in Algorithm 5 and execute $O(1)$ operations. Therefore, if we run Algorithm 5 for every airport pair, the overall complexity is $O(K|E|)$.

6. AHP-based reachability evaluation

Now, we implement Analytic Hierarchy Process (AHP) to convert the reachability feature into numerical values for all airports to measure their service qualities precisely. Fig. 6 shows our AHP model with 2 layers and 8 indices (defined in Sec. 3). At first, we need to normalize these data into $[0, 1]$ range. The normalization methods are summarized in Table 1. Table 2 shows some sample data triples after the normalization.

We then survey and get expert grading matrix for these indices. Table 3 is an example matrix for centrality with 5 indices. Next, we use the matrix to grade and calculate the weights of these features, with properties values and Consistency

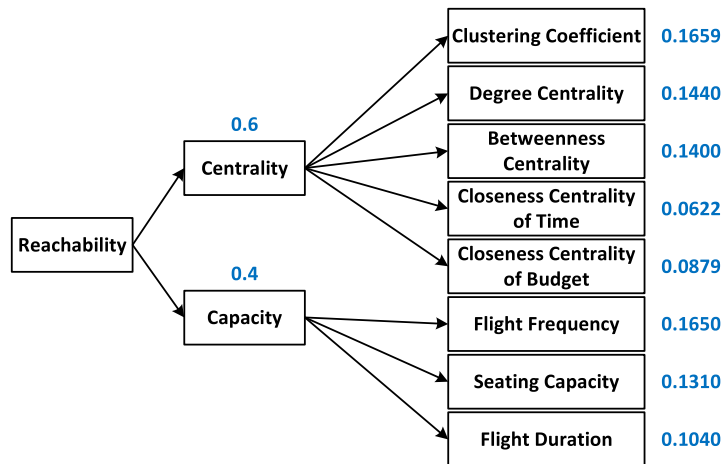


Fig. 6. AHP properties for reachability evaluation. (For interpretation of the references to color in this figure, the reader is referred to the web version of this article.)

Table 1
Normalization methods for indices.

Index	Normalization methods
Clustering Coefficient (CC)	min–max normalization
Degree Centrality (DC)	min–max normalization
Betweenness Centrality (BC)	min–max normalization
Closeness Centrality of Time (CCT)	reciprocal + min–max normalization
Closeness Centrality of Budget (CCB)	reciprocal + min–max normalization
Flight Frequency (FF)	log of the minimum + min–max normalization
Seating Capacity (SC)	min–max normalization
Flight Duration (FD)	reciprocal + min–max normalization

Table 2
Data after normalization.

IATA	City	CC	DC	BC	CCT	CCB	FF	SC	FD
XIY	Xi'an	0.298	0.811	1.000	0.920	0.996	0.932	0.781	0.175
PEK	Beijing	0.240	1.000	0.632	1.000	0.792	1.000	1.000	0.063
CTU	Chengdu	0.312	0.803	0.527	0.992	0.821	0.928	0.861	0.122
CKG	Chongqing	0.338	0.724	0.544	0.923	0.979	0.919	0.767	0.164
KMG	Kunming	0.364	0.717	0.557	0.856	0.814	0.945	0.716	0.177
.	.	.	.	.	.	.	.	.	.
SHF	Shihezi	0.000	0.016	0.004	0.172	0.419	0.095	0.498	0.000
EJN	Ejina	0.000	0.008	0.000	0.216	0.165	0.116	0.081	0.221
RHT	Alxa Youqi	0.000	0.008	0.000	0.000	0.000	0.000	0.000	0.606

Table 3
Pairwise comparison matrix for centrality.

	CC	DC	BC	CCT	CCB	Properties	CR
CC	1	1	2	3	1	0.2765	
DC	1	1	1	2	2	0.2400	
BC	1/2	1	1	3	2	0.2333	0.0469
CCT	1/3	1/2	1/2	1	1	0.1037	
CCB	1	1/2	1/2	1	1	0.1465	

Ratio (CR). Since here $CR = CI/RI = 0.0469 < 0.1$, meaning the value of Consistency Ratio is smaller or equal to 10%, the inconsistency is acceptable. Finally, we get all the properties for indices, shown in Fig. 6 as blue numbers next to each block.

Finally, we calculate the reachability value of all the airports. Table 4 is a sample of accurate results and we also provide a visualization map to show the distribution of the reachability in Fig. 7.

Obviously, in Fig. 7 a lighter and bigger circle represents an airport with higher reachability. It is apparently that the top two airports are Xi'an and Beijing. It is seemingly surprising that Xi'an airport is with higher reachability than Beijing, but

Table 4
Reachability results.

IATA	City	CC	DC	BC	CCT	CCB	FF	SC	FD	R
XIY	Xi'an	0.298	0.811	1.000	0.920	0.996	0.932	0.781	0.175	0.725
PEK	Beijing	0.240	1.000	0.632	1.000	0.792	1.000	1.000	0.063	0.707
CTU	Chengdu	0.312	0.803	0.527	0.992	0.821	0.928	0.861	0.122	0.654
CKG	Chongqing	0.338	0.724	0.544	0.923	0.979	0.919	0.767	0.164	0.649
KMG	Kunming	0.364	0.717	0.557	0.856	0.814	0.945	0.716	0.177	0.635
...	...	...	...	...	...	...	...	...	...	...
NGQ	Gunsu	0.000	0.016	0.000	0.207	0.114	0.150	0.540	0.343	0.156
RKZ	Shigatse	0.000	0.008	0.000	0.270	0.205	0.116	0.535	0.115	0.137
SHF	Shihezi	0.000	0.016	0.004	0.172	0.419	0.095	0.498	0.000	0.131
EJN	Ejina	0.000	0.008	0.000	0.216	0.165	0.116	0.081	0.221	0.082
RHT	Alxa Youqi	0.000	0.008	0.000	0.000	0.000	0.000	0.000	0.606	0.064

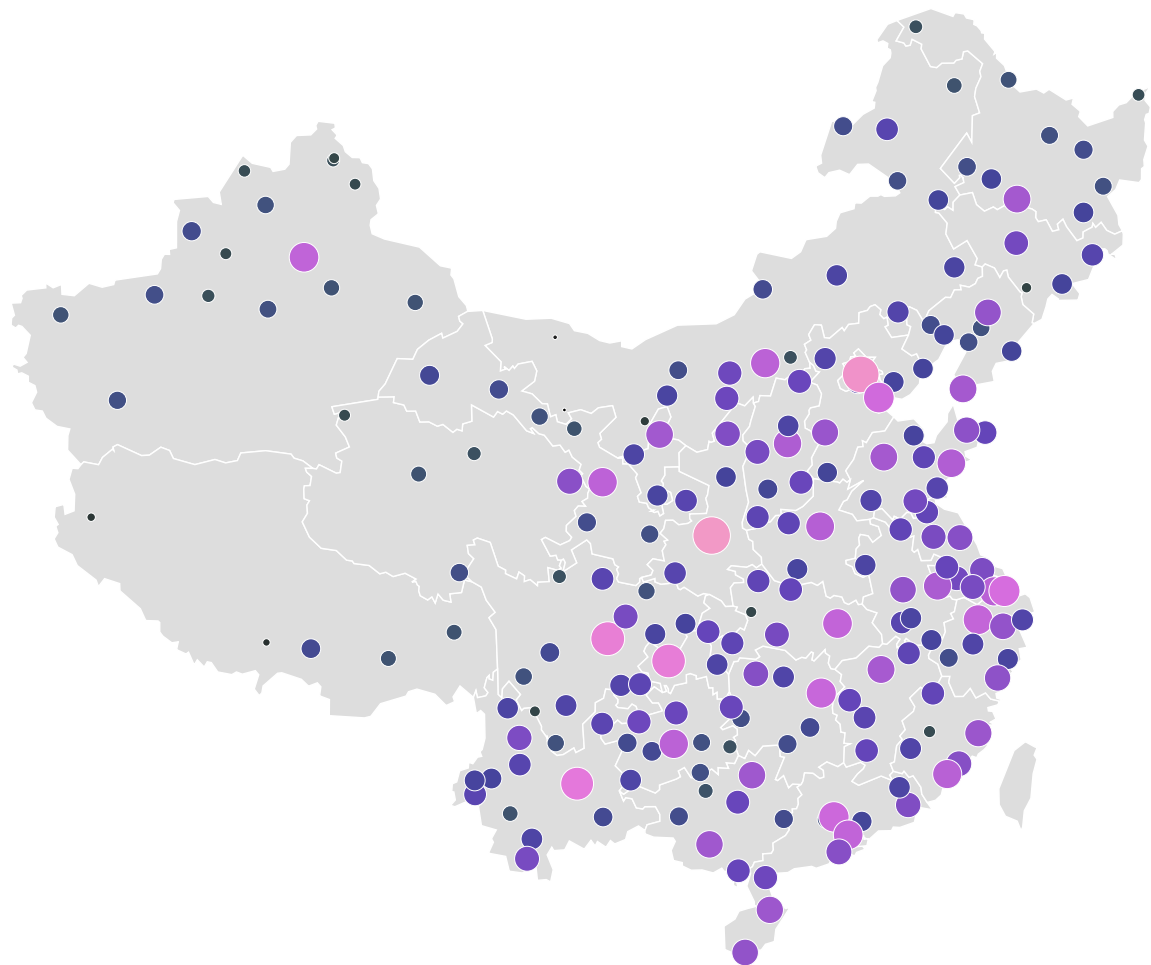


Fig. 7. Visualization results for reachability of airports in China.

noting that what we discuss about is reachability. Considering the location of Xi'an in China and the fact that almost 15% shortest paths passing through it, the conclusion is meaningful to some extent.

7. Conclusion

In this paper, we take China airline network as a representative and try to discuss the reachability issue for each airport respectively, which could reflect its regional connectivity level and service quality of civil aviation. More specifically, we evaluate reachability through many features including node degree, closeness, betweenness, etc. To get some feature values, we design a fast Dijkstra-based all-pair shortest path algorithm with both time and budget requirements, using priority

Queue and Fenwick Tree to improve the time efficiency. Furthermore, we reduce the dummy edges of the duplicated graph and provide faster algorithm with its time complexity analysis.

Finally, we implement Analytic Hierarchy Process (AHP) to convert the reachability feature as numerical values for all airports to measure their service qualities precisely. We not only give a algorithm design and an AHP evaluation, but also prove theoretically the correctness of our method. Such methodology can be easily extended to other airline networks or any arbitrary network with transfer and duration constraints. Our results for China domestic airline network with 210 nodes and 69,160 flight routes will definitely become guide and reference to airline companies and civil aviation administration for their further development and management.

Acknowledgements

This work has been supported in part by Program of International S&T Cooperation (2016YFE0100300), the China 973 Project (2014CB340303), China NSF Projects (Nos. 61571441, 61672353, 61472252), and CCF-Tencent Open Research Fund RAGR20170114.

References

- [1] U. Brandes, A faster algorithm for betweenness centrality, *J. Math. Sociol.* 25 (2) (2001) 163–177.
- [2] C.P. Barros, N. Peypoch, An evaluation of European airlines's operational performance, *Int. J. Prod. Econ.* 122 (2) (2009) 525–533.
- [3] S.-H. Tsaur, T.-Y. Chang, C.-H. Yen, The evaluation of airline service quality by fuzzy MCDM, *Tour. Manag.* 23 (2) (2002) 107–115.
- [4] M.-M. Yu, Assessment of airport performance using the SBM-NDEA model, *Omega* 38 (6) (2010) 440–452.
- [5] B.D. Bowen, D.E. Headley, Evaluation of the US airline industry: the airline quality rating, *Aviation Technology Faculty and Staff Publications* 7 (11) (2013), Paper 7.
- [6] N. Kourtellis, T. Alahakoon, R. Simha, A. Iamnitich, R. Tripathi, Identifying high betweenness centrality nodes in large social networks, *Soc. Netw. Anal. Min.* 3 (4) (2013) 899–914.
- [7] S.-H. Lee, J.-Y. Choi, S.-H. Yoo, Y.-G. Oh, Evaluating spatial centrality for integrated tourism management in rural areas using GIS and network analysis, *Tour. Manag.* 34 (2013) 14–24.
- [8] R. Guimera, L.A.N. Amaral, Modeling the world-wide airport network, *Eur. Phys. J., B Cond. Matter Phys.* 38 (2) (2004) 381–385.
- [9] N.T. Moungla, L. Létocart, A. Nagih, An improving dynamic programming algorithm to solve the shortest path problem with time windows, *Electron. Notes Discrete Math.* 36 (2010) 931–938.
- [10] T. Zhang, Y. Yin, J. Li, Shortest path problems with left-side time windows, *Optim. Lett.* 6 (8) (2012) 1935–1943.
- [11] Y.-L. Chen, K. Tang, Minimum time paths in a network with mixed time constraints, *Comput. Oper. Res.* 25 (10) (1998) 793–805.
- [12] R.K. Ahuja, J.B. Orlin, S. Pallottino, M.G. Scutellà, Minimum time and minimum cost-path problems in street networks with periodic traffic lights, *Transp. Sci.* 36 (3) (2002) 326–336.
- [13] K.N. Androutsopoulos, K.G. Zografos, Solving the k-shortest path problem with time windows in a time varying network, *Oper. Res. Lett.* 36 (6) (2008) 692–695.
- [14] X.-B. Hu, M.-K. Zhang, J.-Q. Liao, H.-L. Zhang, A ripple-spreading algorithm to calculate the k shortest paths in a network with time-windows, in: *Natural Computation, Fuzzy Systems and Knowledge Discovery*, 2016, pp. 376–382.
- [15] D.J. Watts, S.H. Strogatz, Collective dynamics of small-world networks, *Nature* 393 (6684) (1998) 440–442.
- [16] L.C. Freeman, Centrality in social networks conceptual clarification, *Soc. Netw.* 1 (3) (1978) 215–239.
- [17] L.C. Freeman, A set of measures of centrality based on betweenness, *Sociometry* (1977) 35–41.
- [18] G. Sabidussi, The centrality index of a graph, *Psychometrika* 31 (4) (1966) 581–603.
- [19] E.W. Dijkstra, A note on two problems in connexion with graphs, *Numer. Math.* 1 (1) (1959) 269–271.
- [20] M.L. Fredman, R.E. Tarjan, Fibonacci heaps and their uses in improved network optimization algorithms, *J. ACM* 34 (3) (1987) 596–615.
- [21] P.M. Fenwick, A new data structure for cumulative frequency tables, *Softw. Pract. Exp.* 24 (3) (1994) 327–336.